KB-014: Syntax Theme Mapping for Exporters

Executive Summary

Chunk 1 (Complete): Trace document identifies that `syntax_theme` exists in `RuntimeOptions` but is never forwarded to PDF/HTML exporters.

Chunk 2 (Complete): Design decisions document (KB-014b-design.md) with 6 decisions on API signatures, PDF/HTML approaches, and theme mapping.

Chunk 3 (Complete): Implementation of `highlight_code_html()`, `highlight_code_pdf()`, and exporter updates.

Chunk 4 (Complete): All 198 tests pass with syntax highlighting working in both PDF and HTML exports.

Implementation Notes

`highlight_code_html()`

- Uses Pygments HtmlFormatter with style=syntax_theme
- Returns <pre>......</pre> HTML

`highlight_code_pdf()`

- Returns list of dicts: {"text": str, "rgb": dict[str, int]}
- Groups consecutive tokens with same color
- Walks up Pygments token type hierarchy for fallback colors

`_extract_all_token_colors()`

- Merges theme colors with syntax theme colors
- Syntax theme takes precedence
- Handles Pygments style definitions with modifiers (italic, bold)

Test Results

- 198 tests pass
- PDF export verified with test_highlight.md
- HTML export verified with test_highlight.md

Chunk 3 (Complete): Implementation of `highlight_code_html()`, `highlight_code_pdf()`, and exporter updates.

Chunk 4 (Complete): All 198 tests pass with syntax highlighting working in both PDF and HTML exports.

Implementation Notes

`highlight_code_html()`

- Uses Pygments HtmlFormatter with style=syntax_theme
- Returns <pre>......</pre> HTML

`highlight_code_pdf()`

- Returns list of dicts: {"text": str, "rgb": dict[str, int]}
- Groups consecutive tokens with same color
- Walks up Pygments token type hierarchy for fallback colors

`_extract_all_token_colors()`

- Merges theme colors with syntax theme colors
- Syntax theme takes precedence
- Handles Pygments style definitions with modifiers (italic, bold)

Test Results

- 198 tests pass
- PDF export verified with test_highlight.md

KB-014: Syntax Theme Mapping for Exporters

Executive Summary

Chunk 1 (Complete): Trace document identifies that `syntax_theme` exists in `RuntimeOptions` but is never forwarded to PDF/HTML exporters.

Chunk 2 (Complete): Design decisions document (KB-014b-design.md) with 6 decisions on API signatures, PDF/HTML approaches, and theme mapping.

Chunk 3 (Complete): Implementation of `highlight_code_html()`, `highlight_code_pdf()`, and exporter updates.

Chunk 4 (Complete): All 198 tests pass with syntax highlighting working in both PDF and HTML exports.

Implementation Notes

`highlight_code_html()`

- Uses Pygments HtmlFormatter with style=syntax_theme
- Returns <pre>......</pre> HTML

`highlight_code_pdf()`

- Returns list of dicts: {"text": str, "rgb": dict[str, int]}
- Groups consecutive tokens with same color
- Walks up Pygments token type hierarchy for fallback colors

`_extract_all_token_colors()`

- Merges theme colors with syntax theme colors
- Syntax theme takes precedence
- Handles Pygments style definitions with modifiers (italic, bold)

Test Results

- 198 tests pass
- PDF export verified with test_highlight.md
- HTML export verified with test_highlight.md

---

1. CLI Entry Point (`mdutil/cli.py`)

1.1 `--syntax-theme` flag definition (line 100-105)

```
arg_parser.add_argument(
    "--syntax-theme",
    choices=syntax_theme_names(),
    default=None,
    help="Choose Pygments syntax style for code highlighting",
)
```

- syntax_theme_names() returns Pygments styles sorted alphabetically (line 179-183 in themes.py)
- Uses pygments.styles.get_all_styles() as source of truth for valid names
- Default: None (means "use config value" or "use 'default'")

1.2 RuntimeOptions TypedDict (line 65-79)

```
class RuntimeOptions(TypedDict):
    theme: str
    theme_file: str | None
    syntax_theme: str         # ← already defined here
    line_numbers: bool
    quiet: bool
    status_bar_normal: str | None
    status_bar_insert: str | None
    export_format: str
    export_output_dir: str | None
    pdf_paper_size: str
    pdf_margin_top: int
    pdf_margin_bottom: int
    pdf_margin_left: int
    pdf_margin_right: int
```

Note: `syntax_theme` is already part of `RuntimeOptions` and is passed to the terminal renderer but NOT forwarded to exporters.

1.3 Validation and resolution (line 332-355)

```
syntax_theme = cast(
    str,
    args.syntax_theme if args.syntax_theme is not None else config["syntax_theme"],
)
if syntax_theme not in syntax_theme_names():
    valid = ", ".join(syntax_theme_names())
    arg_parser.error(f"invalid syntax theme in configuration: {syntax_theme!r} (choose from {valid})")
```

- Validates against syntax_theme_names() (Pygments styles)
- Falls back to config value, which is set in config.py:77-83 from [syntax_theme] section or DEFAULTS["syntax_theme"] = "default"

1.4 The divergence: `_export_single()` (line 241-291)

```
def _export_single(export_format: str, args: argparse.Namespace, parsed: list|dict], runt
```

```
ime: RuntimeOptions) -> int:
    exporter = PdfExporter() if export_format == "pdf" else HtmlExporter()
    theme = load_theme(runtime["theme"], runtime["theme_file"])
    options: dict[str, Any] = {}

    if export_format == "pdf":
        options["pdf_paper_size"] = runtime.get("pdf_paper_size", "A4")
        # ... pdf margins ...
        options["pdf_bookmarks"] = True
    elif export_format == "html":
        # ... custom CSS handling ...

    # ⚠ BUG: syntax_theme is NOT added to options
    # options["syntax_theme"] is never set
    export_output = exporter.render(parsed, theme=theme, options=options)
```

Key finding: The `options` dict passed to `exporter.render()` contains PDF-specific options but no `syntax_theme`. The exporter has no way to access the chosen Pygments style.

---

2. Terminal Renderer (`mdutil/renderer.py`) — Working Path

2.1 Flow (line 16-36)

```
def render(
    parsed_content: list[dict[str, Any]],
    theme: str = DEFAULT_THEME,
    theme_file: str | None = None,
    syntax_theme: str = "default",  # ← syntax_theme flows here
    line_numbers: bool = False,
    quiet: bool = False,
) -> str:
    selected_theme = load_theme(theme, theme_file)
    result_lines: list[str] = []
    for token in parsed_content:
        result_lines.extend(_render_token(token, selected_theme, syntax_theme))
    ...
```

2.2 Code block rendering (line 79-82)

```
def _render_code(token: dict[str, Any], theme: dict[str, Any], syntax_theme: str = "default") -> list[str]:
    code = str(token.get("content", ""))
    language = str(token.get("language") or "")
```

```
    return highlight_code(code, language, theme, syntax_theme=syntax_theme).split("\n")
```

This path works correctly: `syntax_theme` is passed to `highlight_code()` which uses it to build a Pygments Style.

---

3. Highlighter (`mdutil/syntax_highlighter.py`) — Shared Core

3.1 `highlight_code()` (line 29-60) — Terminal path

```
def highlight_code(
    code: str,
    language: str = "",
    theme: dict[str, Any] | None = None,
    syntax_theme: str = "default",
) -> str:
    lexer_name = language.strip().lower()
    if not lexer_name or lexer_name in _PLAIN_TEXT_LANGUAGE_ALIASES:
        return code

    try:
        lexer = get_lexer_by_name(lexer_name, stripall=False)
    except ClassNotFound:
        return code

    formatter = TerminalTrueColorFormatter(
        style=_style_for_theme(theme, syntax_theme), bg=""
    )
    highlighted = highlight(code, lexer, formatter)
    return highlighted.rstrip("\n")
```

3.2 `_style_for_theme()` (line 63-82)

Merges theme colors (`CODE_COLORS`) with syntax theme colors from Pygments, using syntax theme as precedence.

3.3 `_extract_syntax_theme_colors()` (line 103-120)

Looks up the Pygments style by name and extracts hex colors for each token type.

3.4 `get_syntax_theme_colors()` (line 135-153) — Cached helper

Returns color mappings for a Pygments style by name, with caching via `_style_cache`.

---

4. PDF Exporter (`mdutil/export/pdf.py`) — Needs Fix

4.1 `render()` signature (line 197)

```
def render(self, tokens: list|dict], theme: dict, options: dict) -> bytes:
```

Issue: `options` dict doesn't include `syntax_theme`, so the exporter can't access it.

4.2 `_render_code_block()` (line 346-357) — Current broken state

```
def _render_code_block(self, pdf: FPDF, token: dict) -> None:
    """Render a code block with background."""
    content = token.get("content", "")
    lines = content.split("\n")

    pdf.set_font(self._font_for("mono"), size=9)
    pdf.set_fill_color(240, 240, 240)

    for line in lines:
        pdf.cell(0, 5, line, new_x="LMARGIN", new_y="NEXT", fill=True)

    pdf.ln(5)
```

Issue: Plain text rendering only — no syntax highlighting. Each line is written as a single cell without color variation.

4.3 How it SHOULD work

```
def _render_code_block(self, pdf: FPDF, token: dict) -> None:
    content = token.get("content", "")
    language = token.get("language", "")
    syntax_theme = self._get_syntax_theme()  # ← need to get from options

    # Use highlight_code_pdf() to get (text, rgb) segments
    segments = highlight_code_pdf(content, language, syntax_theme)

    pdf.set_font(self._font_for("mono"), size=9)
    pdf.set_fill_color(240, 240, 240)
```

```
    for segment in segments:
        if segment.get("rgb"):
            r, g, b = segment["rgb"]
            pdf.set_text_color(r, g, b)
        else:
            pdf.set_text_color(0, 0, 0)
        pdf.cell(0, 5, segment["text"], new_x="LMARGIN", new_y="NEXT", fill=True)

    pdf.ln(5)
```

---

5. HTML Exporter (`mdutil/export/html.py`) — Needs Fix

5.1 `render()` signature (line 37)

```
def render(self, tokens: list[dict], theme: dict, options: dict) -> str:
```

Issue: Same `options` dict doesn't include `syntax_theme`.

5.2 `_render_code_block()` (line 295-305) — Current broken state

```
def _render_code_block(self, token: dict) -> str:
    """Render a code block."""
    content = token.get("content", "")
    language = token.get("language", "")

    # Escape HTML entities
    content = content.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")

    if language:
        return f'<pre><code class="language-{language}">{content}</code></pre>'
    return f"<pre><code>{content}</code></pre>"
```

Issue: HTML-escaped plain text only — no Pygments highlighting. The 'pre' 'code' structure is correct, but the content is just escaped text without color classes.

5.3 How it SHOULD work

```
def _render_code_block(self, token: dict) -> str:
    content = token.get("content", "")
    language = token.get("language", "")
```

```
    syntax_theme = self._get_syntax_theme()  # ← need to get from options

    # Use highlight_code_html() to get Pygments HTML
    highlighted_html = highlight_code_html(content, language, syntax_theme)

    if language:
        return f'<pre><code class="language-{language}">{highlighted_html}</code></pre>'
    return f"<pre><code>{highlighted_html}</code></pre>"
```

---

6. Theme Configuration (`mdutil/themes.py`)

6.1 `syntax_theme_names()` (line 179-183)

```
def syntax_theme_names() -> list[str]:
    """Return valid syntax (Pygments) style names for CLI validation/help."""
    from pygments.styles import get_all_styles
    return sorted(get_all_styles())
```

Source of truth for valid Pygments styles. Currently ~49 styles.

6.2 `THEME_SYNTAX_THEMES` (line 41-47)

```
THEME_SYNTAX_THEMES: dict[str, str] = {
    "colored": "default",
    "dracula": "dracula",
    "high-contrast": "bw",
    "one-dark": "one-dark",
    "onedark": "one-dark",
}
```

Note: These map built-in themes to Pygments styles. This is the source of truth for "which Pygments style should I use for this theme?".

6.3 Built-in themes (`line 163-169`)

```
BUILT_IN_THEMES: dict[str, dict[str, Any]] = {
    "colored": COLORED,
    "dracula": DRACULA,
    "high-contrast": HIGH_CONTRAST,
    "one-dark": ONE_DARK,
```

```
    "onedark': ONE_DARK,
}
```

Each theme has a `"syntax"` key (line 60, 94, 127, 160) set to `"default"` — this is the fallback.

---

7. Summary of Required Changes

7.1 CLI (`mdutil/cli.py`)

1. Add options["syntax_theme"] = runtime["syntax_theme"] in _export_single() (line 272)
2. Add options["theme"] = theme (already partially present for PDF, need for HTML)

7.2 Highlighter (`mdutil/syntax_highlighter.py`)

1. Add highlight_code_html(code, language, syntax_theme) — returns Pygments HTML
2. Add highlight_code_pdf(code, language, theme, syntax_theme) — returns list of (text, rgb_dict) segments

7.3 PDF Exporter (`mdutil/export/pdf.py`)

1. Update _render_code_block() to use highlight_code_pdf() and iterate segments with pdf.set_text_color()

7.4 HTML Exporter (`mdutil/export/html.py`)

1. Update _render_code_block() to use highlight_code_html()

7.5 Tests

1. Add regression tests for both exporters

---

8. Edge Cases to Handle

Case	Terminal (working)	PDF/HTML (needs fix)

Empty code block	Returns empty string	Should return empty string
No language	Plain text, no error	Plain text, no error
Unknown language	Plain text fallback	Plain text fallback
Unicode in code	Works with ANSI	Must work with fpdf2 unicode fonts
Very long lines	Works	Must not break fpdf2 rendering
Inline formatting in code	Pygments doesn't handle markdown inline	Expected: plain tokens only

---

9. Test Plan

Unit tests (`tests/test_syntax_highlighter.py`)

- test_highlight_code_known_language — Python → ANSI output
- test_highlight_code_unknown_language — unknown → plain text
- test_highlight_code_plain_text_alias — text/txt/plain → plain text
- test_highlight_code_html_highlighted — Python → HTML with spans
- test_highlight_code_pdf_segments — Python → list of (text, rgb) tuples

Integration tests (`tests/test_export.py`)

- test_render_code_block_html_highlighted — Python code → span tags with color classes
- test_render_code_block_pdf_highlighted — Python code → text segments with varying text_color
- test_render_code_block_unknown_language_plain — unknown language → plain text, no error
- test_render_code_block_no_language_plain — no language → plain text, no error
- test_render_code_block_syntax_theme_applied — different syntax themes produce different colors

CLI integration tests

- mdutil --export html --syntax-theme dracula test.md → HTML has highlighted code
- mdutil --export pdf --syntax-theme one-dark test.md → PDF has colored code blocks

---

10. Files to Modify

File	Change
mdutil/cli.py	Forward syntax_theme and theme to exporters
mdutil/syntax_highlighter.py	Add highlight_code_html() and highlight_code_pdf()
mdutil/export/pdf.py	Update _render_code_block() to use new helper
mdutil/export/html.py	Update _render_code_block() to use new helper
tests/test_export.py	Add syntax highlighting regression tests

---

Next Steps

Chunk 1 deliverable: This annotated walkthrough documents the existing pipeline and identifies the gap.

Chunk 2 deliverable: Design decisions on helper signatures, PDF/HTML approaches, and theme mapping.

Chunk 3 deliverable: Implementation of helpers and exporter updates.

Chunk 4 deliverable: Regression tests and verification.